



FASE 0: Preparamos los cimientos (la base de datos)

Antes de abrir Forms Builder, crea estos objetos en tu base de datos. Así el formulario será mucho más fácil de programar.

0.1. Tabla para el historial de acciones

Esta tabla guardará quién, cuándo y por qué se bloqueó o desbloqueó un punto de venta.

sql

```
CREATE TABLE historial_bloqueo_pv (  
    id_historial      NUMBER PRIMARY KEY,  
    id_punto_venta    NUMBER NOT NULL,  
    estado_anterior   VARCHAR2(10),  
    estado_nuevo      VARCHAR2(10),  
    motivo            VARCHAR2(200),  
    usuario_app       VARCHAR2(50),  
    fecha_accion      TIMESTAMP DEFAULT SYSTIMESTAMP,  
    id_cliente        NUMBER,  
    observaciones     VARCHAR2(500)  
);  
  
CREATE SEQUENCE seq_historial_bloqueo START WITH 1 INCREMENT BY 1;
```

0.2. Paquete PL/SQL con toda la lógica

Este paquete hará el trabajo "sucio". El formulario solo lo llamará.

sql

```
CREATE OR REPLACE PACKAGE PKG_BLOQUEO_PV AS  
  
    PROCEDURE pr_bloquear_pv (  
        p_id_pv      IN NUMBER,  
        p_usuario_app IN VARCHAR2,  
        p_motivo      IN VARCHAR2 DEFAULT 'Bloqueo manual'  
    );
```

```

PROCEDURE pr_desbloquear_pv (
    p_id_pv          IN NUMBER,
    p_usuario_app    IN VARCHAR2,
    p_motivo          IN VARCHAR2 DEFAULT 'Desbloqueo manual'
);

```

```

PROCEDURE pr_desbloquear_por_arreglo (
    p_id_pv          IN NUMBER,
    p_id_cliente      IN NUMBER,
    p_monto_acordado IN NUMBER,
    p_num_cuotas      IN NUMBER,
    p_fecha_aceptacion IN DATE,
    p_fecha_vencimiento IN DATE,
    p_usuario_app     IN VARCHAR2,
    p_obs             IN VARCHAR2 DEFAULT NULL
);

```

```

END PKG_BLOQUEO_PV;

```

```

/

```

```

CREATE OR REPLACE PACKAGE BODY PKG_BLOQUEO_PV AS

```

```

-- =====
-- BLOQUEAR UN PUNTO DE VENTA
-- =====

```

```

PROCEDURE pr_bloquear_pv (
    p_id_pv          IN NUMBER,
    p_usuario_app    IN VARCHAR2,
    p_motivo          IN VARCHAR2 DEFAULT 'Bloqueo manual'
) IS
    v_estado punto_venta.estado%TYPE;
BEGIN
    SELECT estado INTO v_estado
    FROM punto_venta WHERE id_punto_venta = p_id_pv FOR UPDATE;

    IF v_estado = 'INACTIVO' THEN
        RAISE_APPLICATION_ERROR(-20010, 'El punto de venta ya está bloqueado.');
```

```

    END IF;

    UPDATE punto_venta SET estado = 'INACTIVO'

```

```

    WHERE id_punto_venta = p_id_pv;

    INSERT INTO historial_bloqueo_pv
    (id_historial, id_punto_venta, estado_anterior, estado_nuevo,
    motivo, usuario_app)
    VALUES
    (seq_historial_bloqueo.NEXTVAL, p_id_pv, 'ACTIVO', 'INACTIVO',
    p_motivo, p_usuario_app);

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END pr_bloquear_pv;

-- =====
-- DESBLOQUEAR UN PUNTO DE VENTA
-- =====

PROCEDURE pr_desbloquear_pv (
    p_id_pv      IN NUMBER,
    p_usuario_app IN VARCHAR2,
    p_motivo      IN VARCHAR2 DEFAULT 'Desbloqueo manual'
) IS
    v_estado punto_venta.estado%TYPE;
BEGIN
    SELECT estado INTO v_estado
    FROM punto_venta WHERE id_punto_venta = p_id_pv FOR UPDATE;

    IF v_estado = 'ACTIVO' THEN
        RAISE_APPLICATION_ERROR(-20011, 'El punto de venta ya está activo.');
```

```

        COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END pr_desbloquear_pv;

-- =====
-- DESBLOQUEAR POR ARREGLO DE PAGO
-- =====

PROCEDURE pr_desbloquear_por_arreglo (
    p_id_pv          IN NUMBER,
    p_id_cliente     IN NUMBER,
    p_monto_acordado IN NUMBER,
    p_num_cuotas     IN NUMBER,
    p_fecha_aceptacion IN DATE,
    p_fecha_vencimiento IN DATE,
    p_usuario_app    IN VARCHAR2,
    p_obs            IN VARCHAR2 DEFAULT NULL
) IS
    v_id_arreglo NUMBER;
BEGIN
    -- Crear el arreglo de pago (asumimos que la tabla arreglo_pago ya existe)
    v_id_arreglo := seq_arreglo_pago.NEXTVAL;

    INSERT INTO arreglo_pago
    (id_arreglo, id_cliente, fecha_acuerdo, monto_acordado,
     num_cuotas, fecha_vencimiento, estado)
    VALUES
    (v_id_arreglo, p_id_cliente, p_fecha_aceptacion, p_monto_acordado,
     p_num_cuotas, p_fecha_vencimiento, 'ACTIVO');

    -- Desbloquear el punto de venta
    UPDATE punto_venta SET estado = 'ACTIVO'
    WHERE id_punto_venta = p_id_pv AND estado = 'INACTIVO';

    -- Registrar en el historial
    INSERT INTO historial_bloqueo_pv
    (id_historial, id_punto_venta, estado_anterior, estado_nuevo,
     motivo, usuario_app, id_cliente, observaciones)
    VALUES
    (seq_historial_bloqueo.NEXTVAL, p_id_pv, 'INACTIVO', 'ACTIVO',
     'Desbloqueo por arreglo de pago ID: ' || v_id_arreglo,

```

```

        p_usuario_app, p_id_cliente, p_obs);

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END pr_desbloquear_por_arreglo;

END PKG_BLOQUEO_PV;
/

```

0.3. Permisos

```

sql

GRANT EXECUTE ON PKG_BLOQUEO_PV TO TU_USUARIO_FORMS;
GRANT SELECT, INSERT, UPDATE ON punto_venta, historial_bloqueo_pv, arreglo_pago TO
TU_USUARIO_FORMS;
GRANT SELECT ON canal, division, territorio, agencia, cliente, servicio, factura TO
TU_USUARIO_FORMS;

```

FASE 1: Creamos el formulario y sus bloques (la estructura)

1. Abre **Forms Builder**. Verás el **Object Navigator** (una ventana con un árbol).
2. Haz clic en **File** → **New** → **Form**. Se crea un módulo llamado **MODULE1**.
3. Guárdalo como **FOR_BLOQUEO_PV.fmb**.

1.1. Bloque de filtros (para buscar puntos de venta)

Este bloque NO estará atado a una tabla. Solo servirá para poner los filtros.

1. En el Object Navigator, haz clic derecho sobre **Data Blocks** → **Create**.
2. Elige **Build a new data block manually** → **Next**.
3. Nombre: **BLK_FILTRO**. **No** marques "Database Data Block". → **Next** → **Finish**.
4. Ahora crea los ítems dentro de **BLK_FILTRO**. Haz clic derecho sobre el bloque → **Create Item**. Crea uno por uno:

Nombre del ítem	Tipo	Para qué sirve
IT_CANAL	List Item	Elegir el canal
IT_DIVISION	List Item	Elegir la división
IT_TERRITORIO	List Item	Elegir el territorio
IT_AGENCIA	List Item	Elegir la agencia
BT_BUSCAR	Push Button	Botón para ejecutar la búsqueda

1.2. Bloque de puntos de venta (la tabla donde se listan)

1.Data Blocks → Create → Next.

2.Nombre: BLK_PV.

3.Selecciona **Yes** para "Based on a table" y elige PUNTO_VENTA.

4.Selecciona las columnas: id_punto_venta, nombre_pv, estado, direccion. → **Finish**.

5.En el Property Palette del bloque BLK_PV, busca la propiedad **Query Data Source Name** y escribe esta consulta (esto permite unir tablas):

```
sql
```

```
(SELECT pv.id_punto_venta, pv.nombre_pv, pv.estado, pv.direccion
FROM punto_venta pv
JOIN agencia a ON a.id_agencia = pv.id_agencia
JOIN territorio t ON t.id_territorio = a.id_territorio
JOIN division d ON d.id_division = t.id_division
WHERE d.nombre_division = 'Clientes de Mi Negocio')
```

6.Cambia la propiedad **Query Data Source Type** a FROM clause query.

1.3. Bloque de botones de acción

1.Data Blocks → Create → Build manually.

2.Nombre: BLK_ACCION.

3.Crea los botones:

Nombre del botón **Etiqueta**

BT_INACTIVAR	Inactivar
BT_ACTIVAR	Activar
BT_ARREGLO	Arreglo de Pago

Nombre del botón	Etiqueta
BT_SALIR	Salir

1.4. Bloque para el arreglo de pago (ventana emergente)

Este bloque tendrá los campos para llenar el arreglo.

1.Data Blocks → Create → Build manually.

2.Nombre: BLK_ARREGLO.

3.Crea estos ítems:

Nombre del ítem	Tipo	Descripción
IT_MONTO	Text Item	Monto total del arreglo
IT_CUOTAS	Text Item	Número de cuotas
IT_FECHA_ACEP	Text Item	Fecha de aceptación
IT_FECHA_VENC	Text Item	Fecha de vencimiento
BT_GUARDAR_ARR	Push Button	Guardar el arreglo
BT_CANCELAR_ARR	Push Button	Cerrar sin guardar



FASE 2: El layout (dibujamos la pantalla)

1.Abre el **Layout Editor** para el canvas principal.

2.Arrastra los ítems desde el Object Navigator hacia el canvas.

3.Para el bloque BLK_ARREGLO, crea una **Window** nueva llamada WIN_ARREGLO y un **Canvas** llamado CVS_ARREGLO. Asigna ese canvas a esa ventana.



FASE 3: La lógica (el código de los triggers)

3.1. Cuando el formulario se abre

Trigger WHEN-NEW-FORM-INSTANCE (a nivel de formulario):

```
sql

BEGIN
    :GLOBAL.usuario_app := 'USUARIO_FORMS'; -- O el usuario real de tu sistema
    PR_CARGAR_LISTAS;
    GO_ITEM('BLK_FILTRO.IT_CANAL');
END;
```

3.2. Program Unit para cargar las listas

Crea un **Program Unit** de tipo **Procedure** llamado PR_CARGAR_LISTAS:

```
sql

PROCEDURE PR_CARGAR_LISTAS IS
BEGIN
    -- Canal
    CLEAR_LIST('BLK_FILTRO.IT_CANAL');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_CANAL', 0, '(Todos)', 0);
    FOR r IN (SELECT id_canal, nombre_canal FROM canal ORDER BY nombre_canal) LOOP
        ADD_LIST_ELEMENT('BLK_FILTRO.IT_CANAL', r.id_canal, r.nombre_canal, r.id_canal);
    END LOOP;

    -- División (solo 'Clientes de Mi Negocio')
    CLEAR_LIST('BLK_FILTRO.IT_DIVISION');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', 0, '(Todas)', 0);
    FOR r IN (SELECT id_division, nombre_division FROM division
              WHERE nombre_division = 'Clientes de Mi Negocio') LOOP
        ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', r.id_division, r.nombre_division,
r.id_division);
    END LOOP;

    -- Territorio y Agencia se Llenan en cascada
```



```

CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);
CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);

:BLK_FILTRO.IT_CANAL := 0;
:BLK_FILTRO.IT_DIVISION := 0;
:BLK_FILTRO.IT_TERRITORIO := 0;
:BLK_FILTRO.IT_AGENCIA := 0;
END;

```

3.3. Cascada de filtros

Trigger WHEN-LIST-CHANGED en IT_CANAL:

```

sql

DECLARE
    v_id_canal NUMBER := :BLK_FILTRO.IT_CANAL;
BEGIN
    CLEAR_LIST('BLK_FILTRO.IT_DIVISION');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', 0, '(Todas)', 0);
    FOR r IN (SELECT id_division, nombre_division FROM division
              WHERE nombre_division = 'Clientes de Mi Negocio'
                AND (v_id_canal = 0 OR id_canal = v_id_canal)) LOOP
        ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', r.id_division, r.nombre_division,
r.id_division);
    END LOOP;
    :BLK_FILTRO.IT_DIVISION := 0;
    CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);
    :BLK_FILTRO.IT_TERRITORIO := 0;
    CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
    :BLK_FILTRO.IT_AGENCIA := 0;
END;

```

Trigger WHEN-LIST-CHANGED en IT_DIVISION:

```

sql

DECLARE

```

```

v_id_div NUMBER := :BLK_FILTRO.IT_DIVISION;
BEGIN
  CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
  ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);
  FOR r IN (SELECT id_territorio, nombre_territorio FROM territorio
            WHERE (v_id_div = 0 OR id_division = v_id_div)) LOOP
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', r.id_territorio, r.nombre_territorio,
r.id_territorio);
  END LOOP;
  :BLK_FILTRO.IT_TERRITORIO := 0;
  CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
  ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
  :BLK_FILTRO.IT_AGENCIA := 0;
END;

```

Trigger WHEN-LIST-CHANGED en IT_TERRITORIO:

```

sql

DECLARE
  v_id_terr NUMBER := :BLK_FILTRO.IT_TERRITORIO;
BEGIN
  CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
  ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
  FOR r IN (SELECT id_agencia, nombre_agencia FROM agencia
            WHERE (v_id_terr = 0 OR id_territorio = v_id_terr)) LOOP
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', r.id_agencia, r.nombre_agencia,
r.id_agencia);
  END LOOP;
  :BLK_FILTRO.IT_AGENCIA := 0;
END;

```

3.4. Botón Buscar

Trigger WHEN-BUTTON-PRESSED en BT_BUSCAR:

```

sql

DECLARE
  v_where VARCHAR2(4000);
BEGIN
  v_where := 'd.nombre_division = ''Clientes de Mi Negocio''';

```

```

IF :BLK_FILTRO.IT_CANAL > 0 THEN
    v_where := v_where || ' AND ca.id_canal = ' || :BLK_FILTRO.IT_CANAL;
END IF;
IF :BLK_FILTRO.IT_DIVISION > 0 THEN
    v_where := v_where || ' AND d.id_division = ' || :BLK_FILTRO.IT_DIVISION;
END IF;
IF :BLK_FILTRO.IT_TERRITORIO > 0 THEN
    v_where := v_where || ' AND t.id_territorio = ' || :BLK_FILTRO.IT_TERRITORIO;
END IF;
IF :BLK_FILTRO.IT_AGENCIA > 0 THEN
    v_where := v_where || ' AND a.id_agencia = ' || :BLK_FILTRO.IT_AGENCIA;
END IF;

SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
GO_BLOCK('BLK_PV');
EXECUTE_QUERY;
END;

```

3.5. Botón Inactivar (Bloquear)

Trigger WHEN-BUTTON-PRESSED en BT_INACTIVAR:

```

sql

DECLARE
    v_id_pv NUMBER := :BLK_PV.ID_PUNTO_VENTA;
BEGIN
    IF v_id_pv IS NULL THEN
        MESSAGE('Debe seleccionar un punto de venta.');
```

```

        RAISE FORM_TRIGGER_FAILURE;
    END IF;

    PKG_BLOQUEO_PV.PR_BLOQUEAR_PV(
        p_id_pv      => v_id_pv,
        p_usuario_app => :GLOBAL.usuario_app,
        p_motivo      => 'Bloqueo manual desde formulario'
    );

    MESSAGE('Punto de venta bloqueado.');
```

```

        :BLK_PV.ESTADO := 'INACTIVO';
    SYNCHRONIZE;
EXCEPTION
    WHEN OTHERS THEN
        MESSAGE('Error: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
END;

```

3.6. Botón Activar (Desbloquear)

Trigger WHEN-BUTTON-PRESSED en BT_ACTIVAR:

```

sql

DECLARE
    v_id_pv NUMBER := :BLK_PV.ID_PUNTO_VENTA;
BEGIN
    IF v_id_pv IS NULL THEN
        MESSAGE('Debe seleccionar un punto de venta.');
```

```

        RAISE FORM_TRIGGER_FAILURE;
    END IF;

    PKG_BLOQUEO_PV.PR_DESBLOQUEAR_PV(
        p_id_pv      => v_id_pv,
        p_usuario_app => :GLOBAL.usuario_app,
        p_motivo      => 'Desbloqueo manual desde formulario'
    );

    MESSAGE('Punto de venta activado.');
```

```

    :BLK_PV.ESTADO := 'ACTIVO';
    SYNCHRONIZE;
EXCEPTION
    WHEN OTHERS THEN
        MESSAGE('Error: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
END;

```

3.7. Botón Arreglo de Pago (abre la ventana emergente)

Trigger WHEN-BUTTON-PRESSED en BT_ARREGLO:

```
sql

BEGIN
    IF :BLK_PV.ID_PUNTO_VENTA IS NULL THEN
        MESSAGE('Debe seleccionar un punto de venta.');
```

RAISE FORM_TRIGGER_FAILURE;

```
    END IF;

    -- Inicializar campos
    :BLK_ARREGLO.IT_MONTO := NULL;
    :BLK_ARREGLO.IT_CUOTAS := 1;
    :BLK_ARREGLO.IT_FECHA_ACEP := SYSDATE;
    :BLK_ARREGLO.IT_FECHA_VENC := ADD_MONTHS(SYSDATE, 3); -- 3 meses después

    GO_ITEM('BLK_ARREGLO.IT_MONTO');
    SHOW_WINDOW('WIN_ARREGLO');
END;
```

3.8. Guardar el arreglo de pago

Trigger WHEN-BUTTON-PRESSED en BT_GUARDAR_ARR:

```
sql

DECLARE
    v_id_pv      NUMBER := :BLK_PV.ID_PUNTO_VENTA;
    v_id_cliente NUMBER; -- Se debe obtener el cliente en mora

BEGIN
    -- Obtener un cliente en mora del punto de venta
    BEGIN
        SELECT c.id_cliente INTO v_id_cliente
            FROM cliente c
            JOIN servicio s ON s.id_cliente = c.id_cliente
            JOIN factura f ON f.id_servicio = s.id_servicio
        WHERE c.id_punto_venta = v_id_pv
            AND f.estado IN ('VENCIDA', 'EN_MORA')
            AND ROWNUM = 1;
```

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        MESSAGE('No se encontró cliente en mora para este PV.');
```

```

        RAISE FORM_TRIGGER_FAILURE;
END;
```

```

PKG_BLOQUEO_PV.PR_DESBLOQUEAR_POR_ARREGLO(
    p_id_pv          => v_id_pv,
    p_id_cliente     => v_id_cliente,
    p_monto_acordado => :BLK_ARREGLO.IT_MONTO,
    p_num_cuotas     => :BLK_ARREGLO.IT_CUOTAS,
    p_fecha_aceptacion => :BLK_ARREGLO.IT_FECHA_ACEP,
    p_fecha_vencimiento => :BLK_ARREGLO.IT_FECHA_VENC,
    p_usuario_app     => :GLOBAL.usuario_app,
    p_obs            => 'Arreglo registrado desde formulario'
);
```

```

MESSAGE('Arreglo de pago guardado. PV desbloqueado.');
```

```

:BLK_PV.ESTADO := 'ACTIVO';
```

```

HIDE_WINDOW('WIN_ARREGLO');
```

```

SYNCHRONIZE;
```

```

EXCEPTION
```

```

    WHEN OTHERS THEN
        MESSAGE('Error: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
```

```

END;
```

3.9. Cancelar arreglo

Trigger WHEN-BUTTON-PRESSED en BT_CANCELAR_ARR:

```

sql
```

```

BEGIN
```

```

    HIDE_WINDOW('WIN_ARREGLO');
```

```

END;
```

3.10. Botón Salir

Trigger WHEN-BUTTON-PRESSED en BT_SALIR:

```
sql  
  
BEGIN  
    EXIT_FORM(NO_VALIDATE);  
END;
```



FASE 4: El desbloqueo automático (el trabajo programado)

Esto no es parte del formulario, sino un **Job de base de datos** que revisa si hay arreglos de pago activos y desbloquea los puntos de venta automáticamente.

Crea este procedimiento en la base de datos:

```
sql  
  
CREATE OR REPLACE PROCEDURE pr_desbloqueo_automatico_arreglos  
IS  
BEGIN  
    FOR r IN (  
        SELECT pv.id_punto_venta, c.id_cliente, ap.id_arreglo  
        FROM punto_venta pv  
        JOIN cliente c ON c.id_punto_venta = pv.id_punto_venta  
        JOIN arreglo_pago ap ON ap.id_cliente = c.id_cliente  
        WHERE pv.estado = 'INACTIVO'  
        AND ap.estado = 'ACTIVO'  
        AND ap.fecha_acuerdo >= TRUNC(SYSDATE) - 1  
    ) LOOP  
        UPDATE punto_venta SET estado = 'ACTIVO'  
        WHERE id_punto_venta = r.id_punto_venta AND estado = 'INACTIVO';  
  
        INSERT INTO historial_bloqueo_pv  
        (id_historial, id_punto_venta, estado_anterior, estado_nuevo,  
        motivo, usuario_app, id_cliente)  
        VALUES
```

```

        (seq_historial_bloqueo.NEXTVAL, r.id_punto_venta, 'INACTIVO', 'ACTIVO',
        'Desbloqueo automático por arreglo ID: ' || r.id_arreglo,
        'SISTEMA', r.id_cliente);

        COMMIT;
    END LOOP;
END;
/

```

Y programa el Job para que se ejecute cada 2 horas:

```

sql

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_DESBLOQUEO_ARR',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PR_DESBLOQUEO_AUTOMATICO_ARREGLOS',
        start_date         => SYSTIMESTAMP,
        repeat_interval    => 'FREQ=HOURLY; INTERVAL=2',
        enabled            => TRUE
    );
END;
/

```

FASE 5: Probamos y conectamos al menú

1. **Compila el formulario:** Ctrl+K.
2. **Ejecútalo:** Ctrl+R.
3. Prueba cada botón.
4. Para agregarlo a tu menú principal, en tu módulo de menú (MENU_PRINCIPAL.mmb), agrega un ítem con la acción:

```

sql

CALL_FORM('FOR_BLOQUEO_PV', HIDE, NO_REPLACE, NO_QUERY_ONLY);

```



Tiempo estimado

Fase	Tiempo
Preparar BD (scripts SQL)	30 min
Crear formulario y bloques	1 hora
Lógica (triggers)	1 hora
Probar y conectar al menú	30 min
Total	3 horas

Con esta guía tienes todo lo necesario. ¡Mucho éxito!